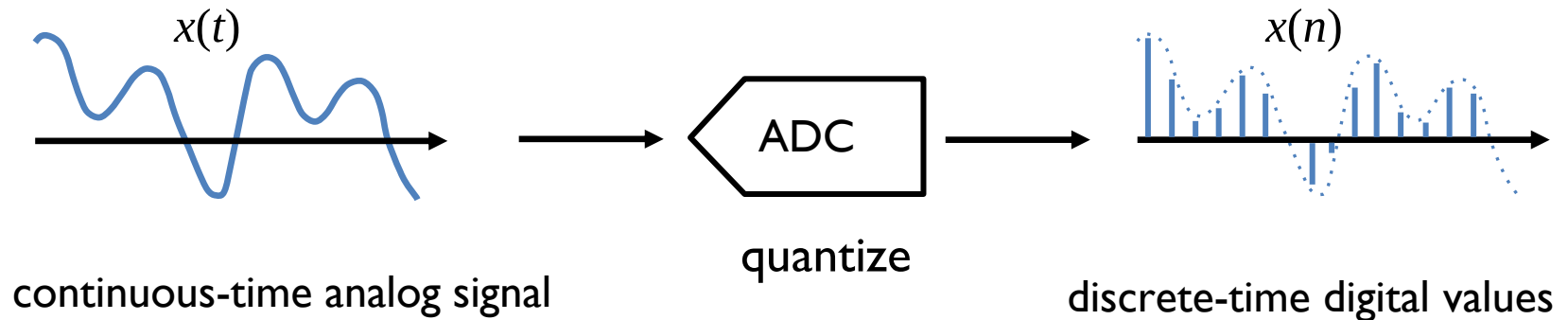


CSE33 I: Microprocessor Interfacing and Embedded Systems

Lecture#14: Analog-to-Digital Converter (ADC) Chapter 20

Analog-to-Digital Converter (ADC)

- ❑ ADC is important almost to all application fields
- ❑ Converts a continuous-time voltage signal within a given range to discrete-time digital values to quantify the voltage's amplitudes



Analog-to-Digital Converter (ADC)

❑ Three performance parameters:

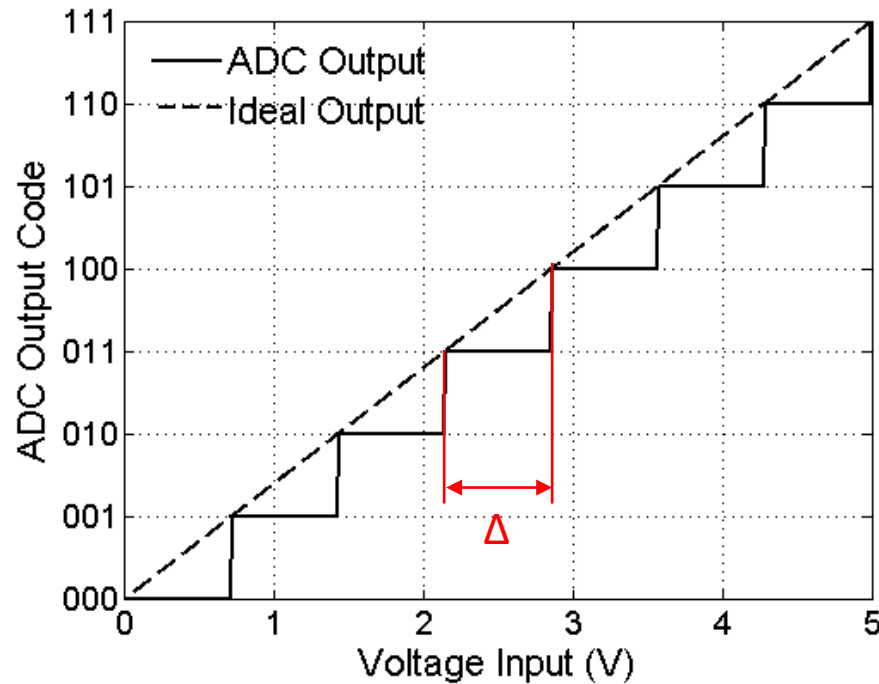
- ❑ sampling rate
- ❑ resolution
- ❑ power dissipation

❑ Many ADC implementations:

- ❑ sigma-delta
- ❑ successive-approximation
- ❑ pipeline

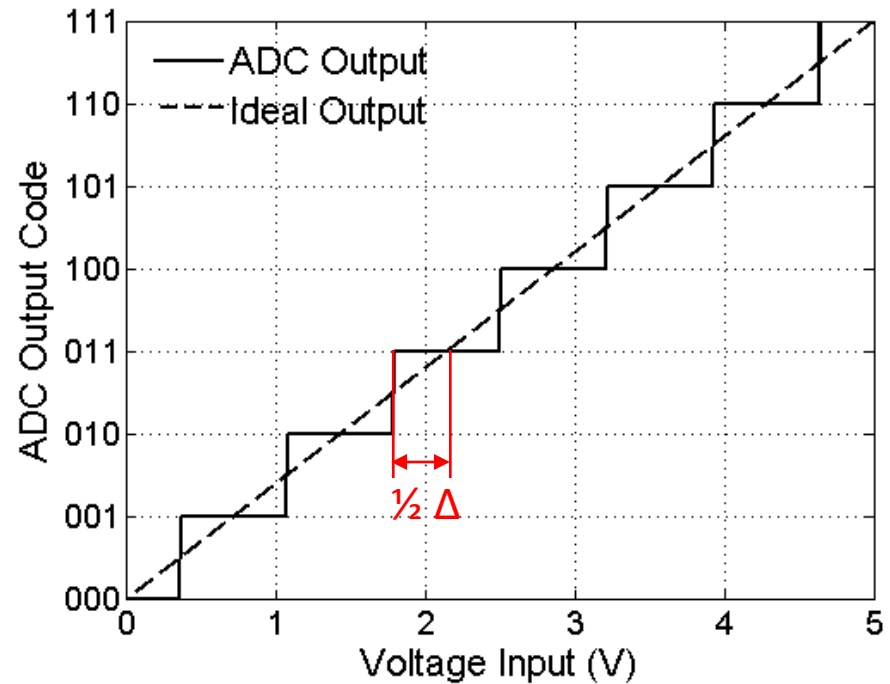
Resolution

- Resolution is determined by number of bits (in binary) to represent an analog input.
- Example of two quantization methods ($N = 3$)



$$\text{Digital Result} = \text{floor}\left(2^3 \times \frac{V}{V_{REF}}\right)$$

$$\text{Max quantization error} = \Delta = V_{REF}/2^3$$



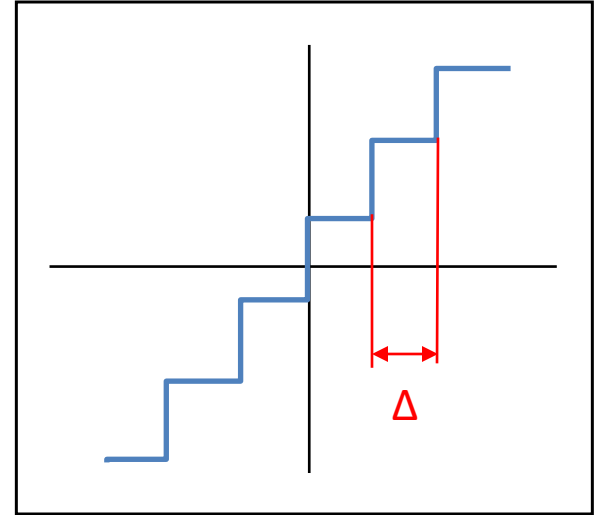
$$\text{Digital Result} = \text{round}\left(2^3 \times \frac{V}{V_{REF}}\right)$$

$$\text{Max quantization error} = \pm \frac{1}{2} \Delta = \pm V_{REF}/2^4$$

$$\text{round}(x) = \text{floor}(x + 0.5)$$

Quantization Error

- For N-bit ADC, it is limited to $\pm \frac{1}{2}\Delta$
- Δ = is the step size of the converter.

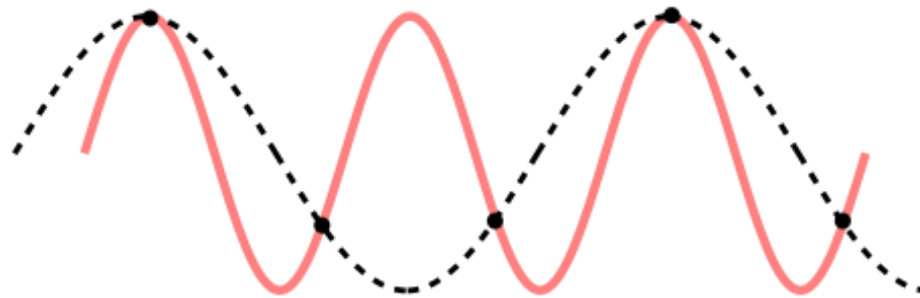


- Example: for 12-bit ADC and input voltage range [0, 3V]

$$\text{Max Quantization Error} = \frac{1}{2}\Delta = \frac{3V}{2 \times 2^{12}} = 0.367\text{mV}$$

Minimum Sampling Rate: Nyquist–Shannon Sampling Theorem

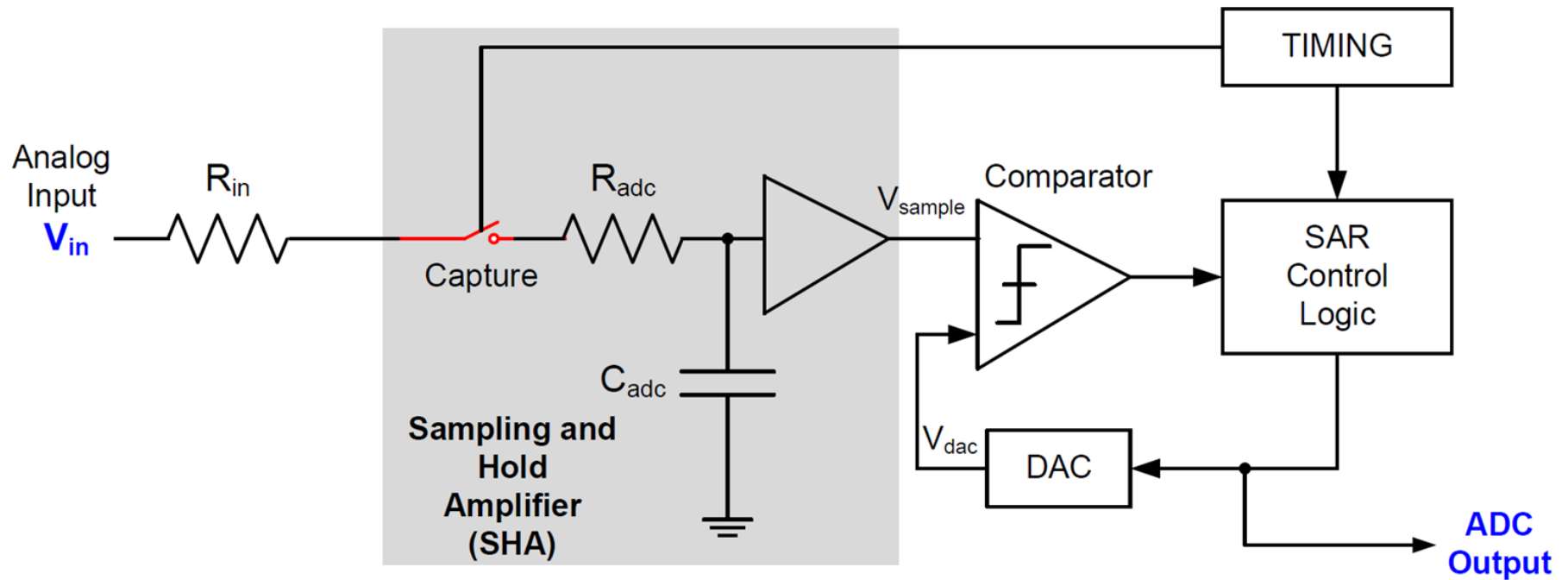
- ❓ In order to be able to reconstruct the analog input signal, the sampling rate should be at least twice the maximum frequency component contained in the input signal
- ❓ Example of two sine waves have the same sampling values. This is called **aliasing**.



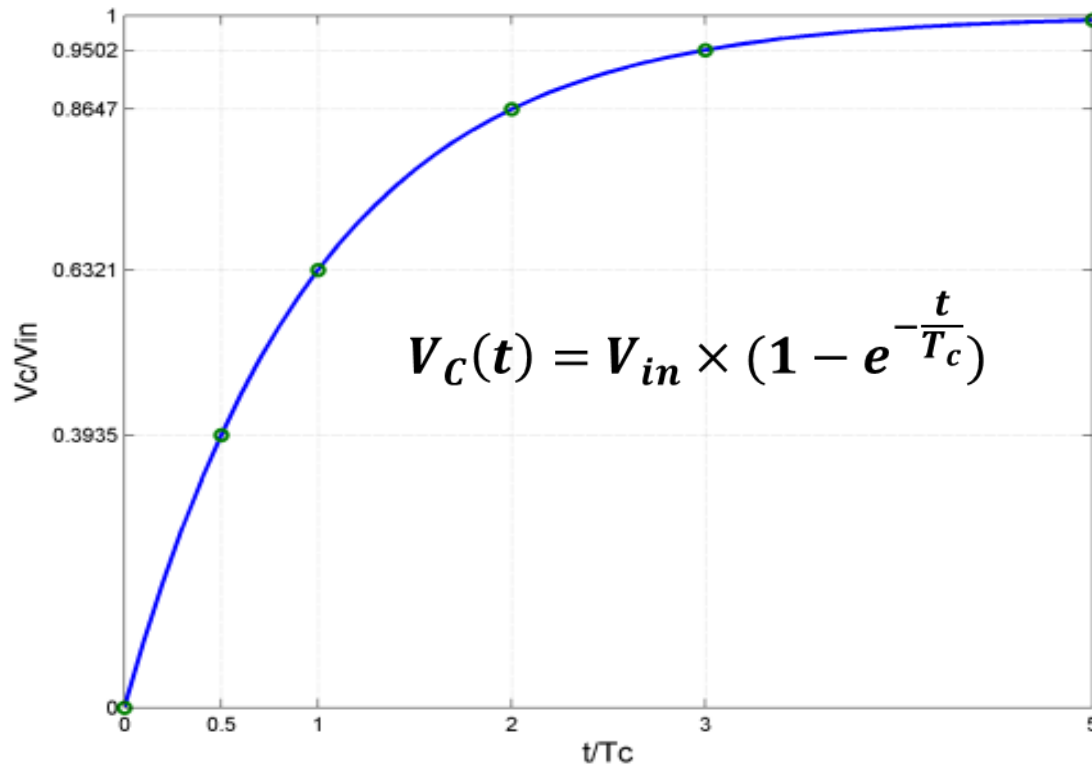
*from
wiki.com*

- ❓ **Antialiasing** (beyond the scope of this course)
 - ❓ **Pre-filtering**: use analog hardware to filtering out high-frequency components and only sampling the low-frequency components. The high-frequency components are ignored.
 - ❓ **Post-filtering**: Oversample continuous signal, then use software to filter out high-frequency components

Successive-approximation (SAR) ADC



Determining Minimum Sampling Time

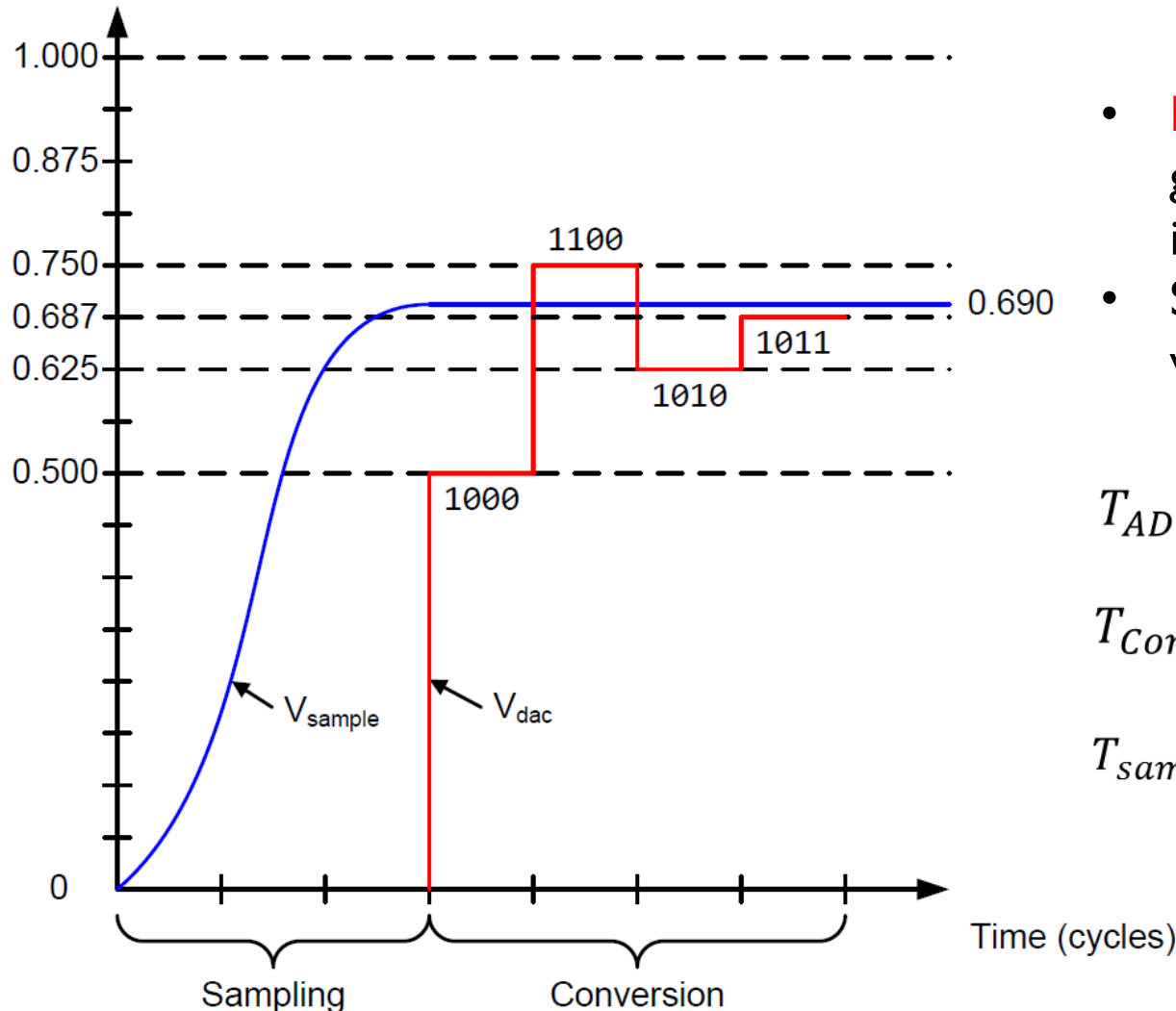


Sampling time is software programmable!

Larger sampling time $\begin{cases} \rightarrow \text{Smaller sampling error} \\ \rightarrow \text{Slower ADC speed} \end{cases}$

Tradeoff

Successive-approximation (SAR) ADC



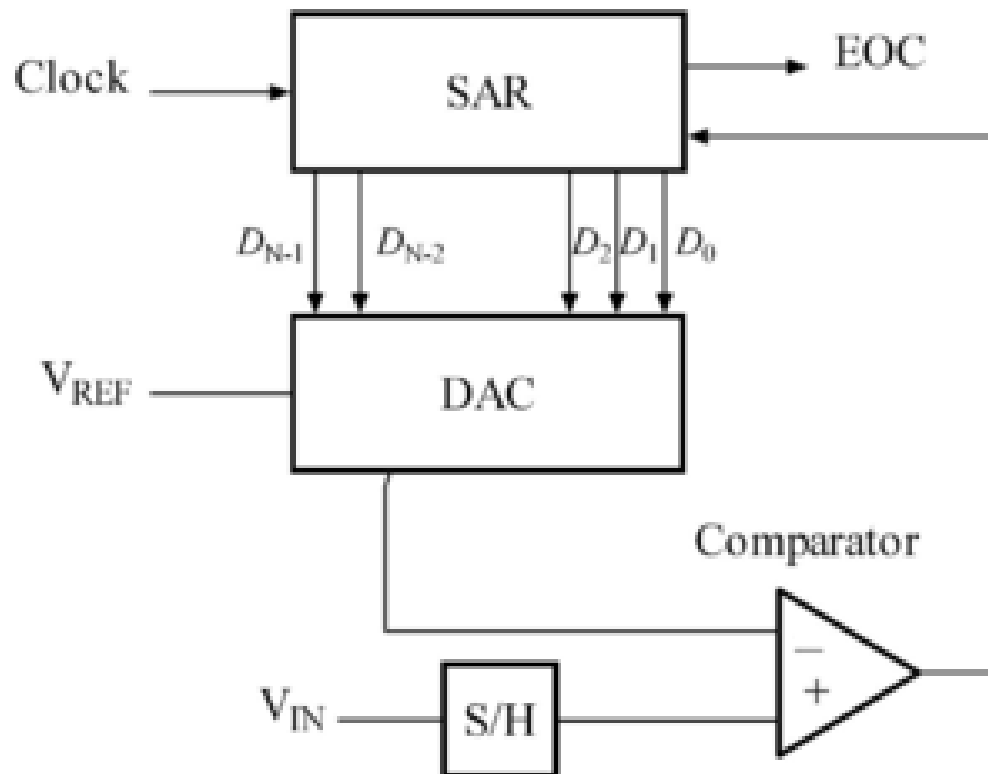
- **Binary search** algorithm to gradually approaches the input voltage
- Settle into $\pm \frac{1}{2}$ LSB bound within the time allowed

$$T_{ADC} = T_{\text{sampling}} + T_{\text{Conversion}}$$

$$T_{\text{Conversion}} = N \times T_{ADC_Clock}$$

T_{sampling} is software configurable

SAR: Successive Approximation Register



The binary weights assigned to each bit, (D_x) starting with the MSB, are 2.5, 1.25, 0.625, 0.3125, 0.15625, 0.078125, 0.0390625, 0.01953125, 0.009765625, 0.0048828125. All of these add up to 4.9951171875, meaning binary 11111111, or one LSB less than 5.

ADC Conversion Time

$$T_{ADC} = T_{sampling} + T_{Conversion}$$

Suppose ADCCLK = 16 MHz and Sampling time = 4 cycles

For 12-bit
ADC

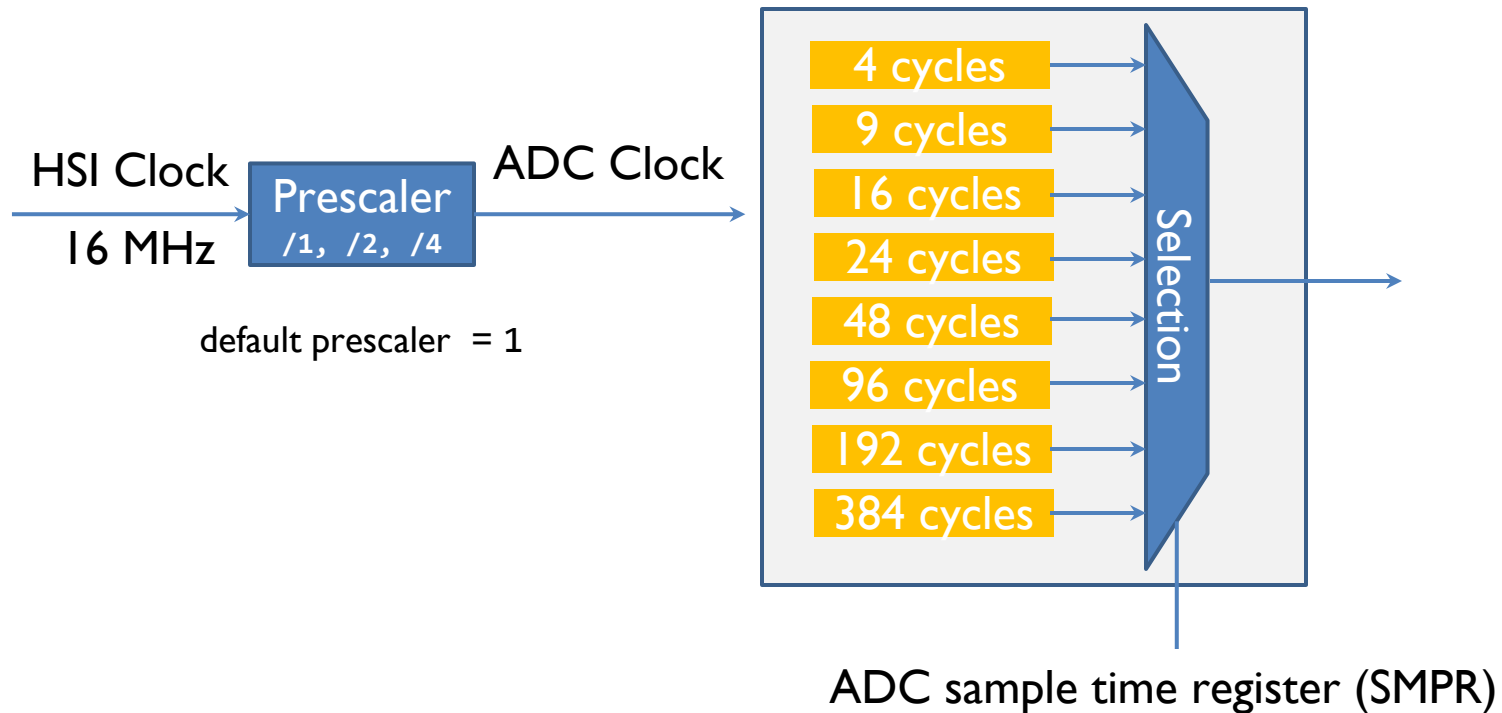
$$T_{ADC} = 4 + 12 = 16 \text{ cycles} = 1\mu s$$

For 6-bit
ADC

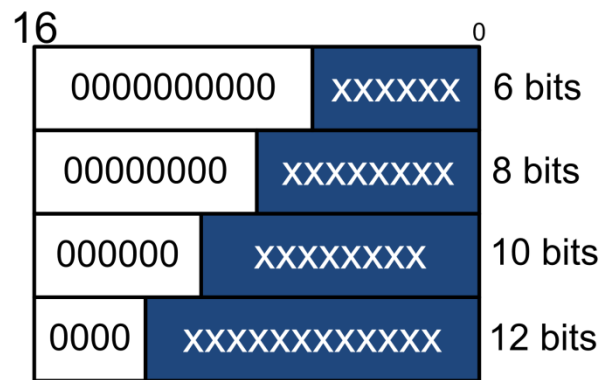
$$T_{ADC} = 4 + 6 = 10 \text{ cycles} = 625ns$$



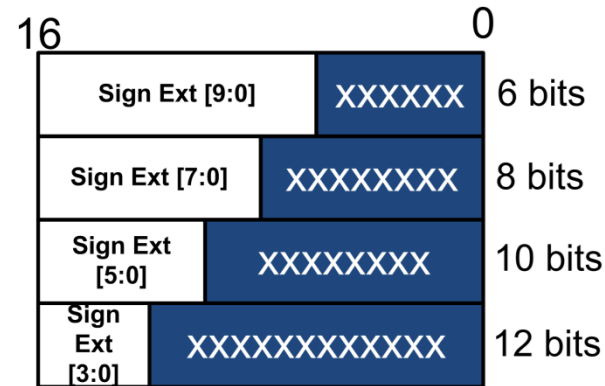
Programming ADC Sampling Time



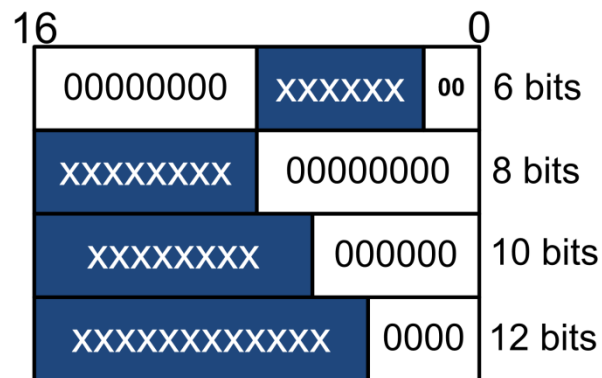
Data Alignment



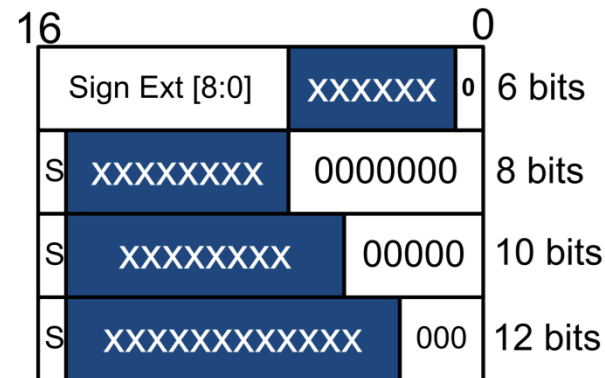
Right alignment for a regular channel



Right alignment for an injected channel



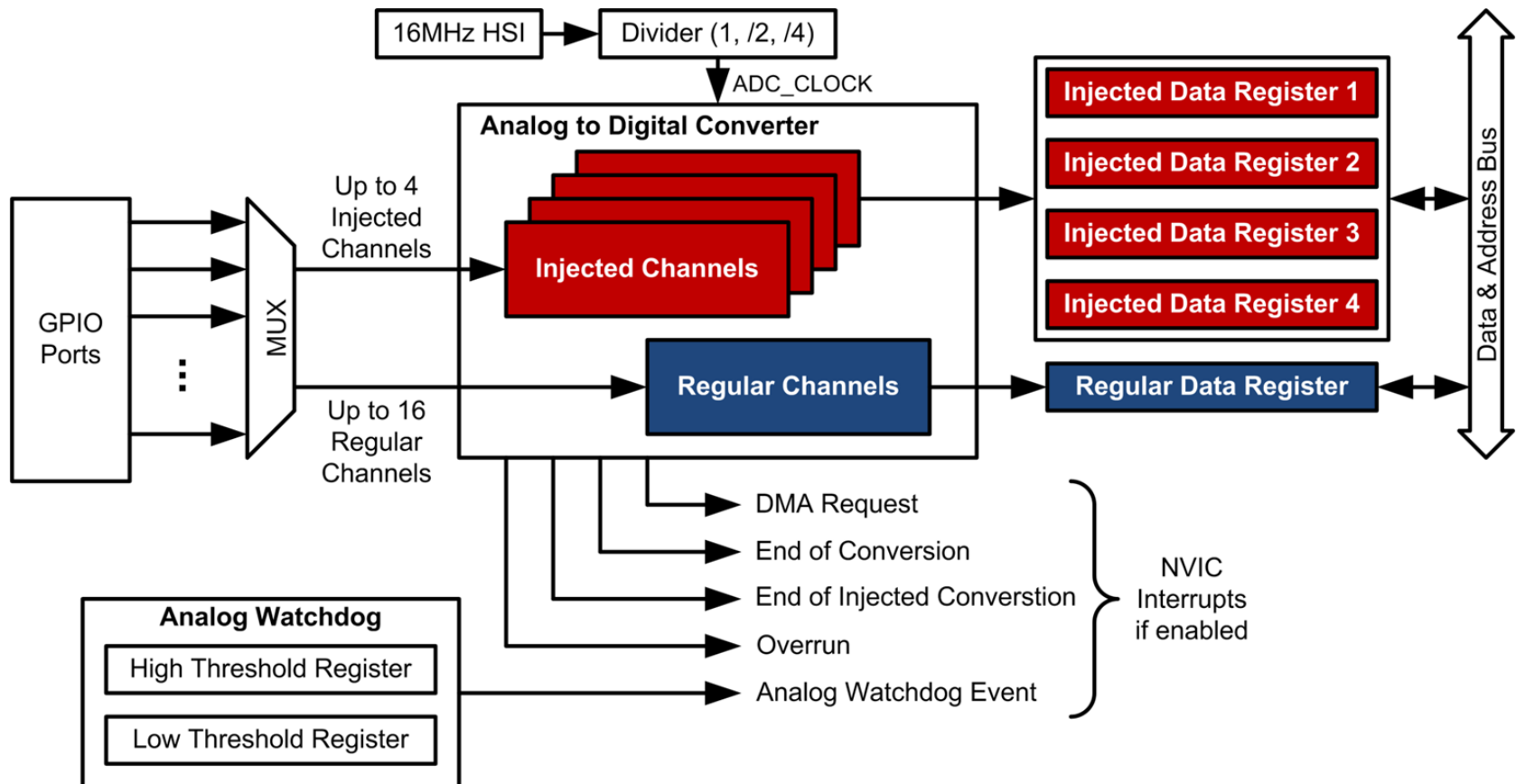
Left alignment for a regular channel



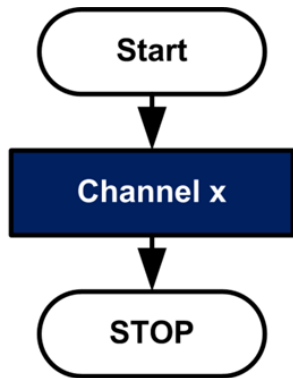
Left alignment for an injected channel

S = Sign bit

ADC: Regular *vs* injected

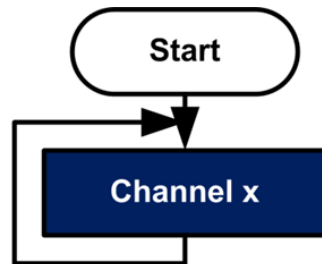


ADC Mode



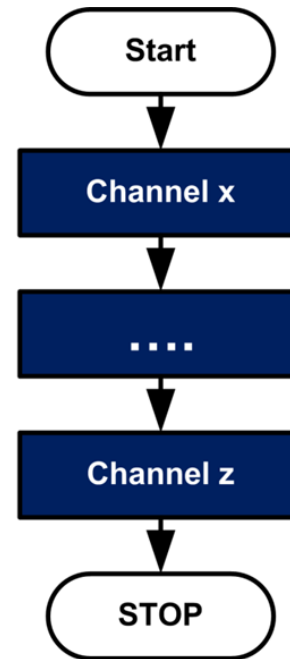
Single Channel, Single Conversion Mode

CONT in ADC_CR2 = 0
SCAN in ADC_CR2 = 0



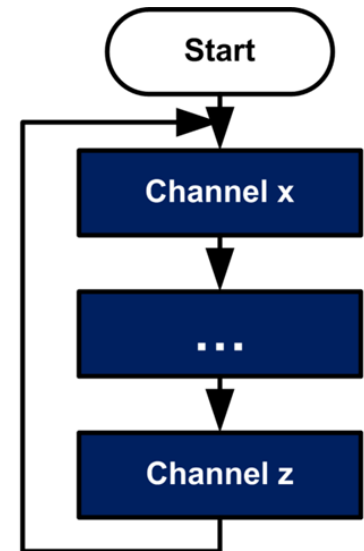
Single Channel, Continuous Conversion Mode

CONT in ADC_CR2 = 1
SCAN in ADC_CR2 = 0



Scan Mode with Single Conversion

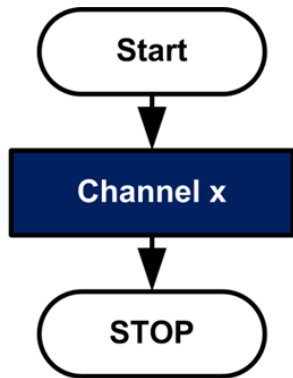
SCAN in ADC_CR2 = 1
CONT in ADC_CR2 = 0



Scan Mode with Continuous Conversion

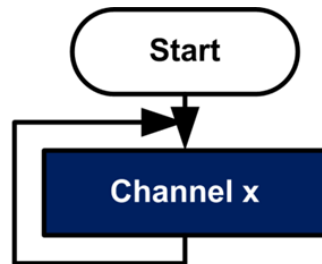
SCAN in ADC_CR2 = 1
CONT in ADC_CR2 = 1

ADC Mode



Single Channel, Single Conversion Mode

CONT in ADC_CR2 = 0
SCAN in ADC_CR2 = 0



Single Channel, Continuous Conversion Mode

CONT in ADC_CR2 = 1
SCAN in ADC_CR2 = 0

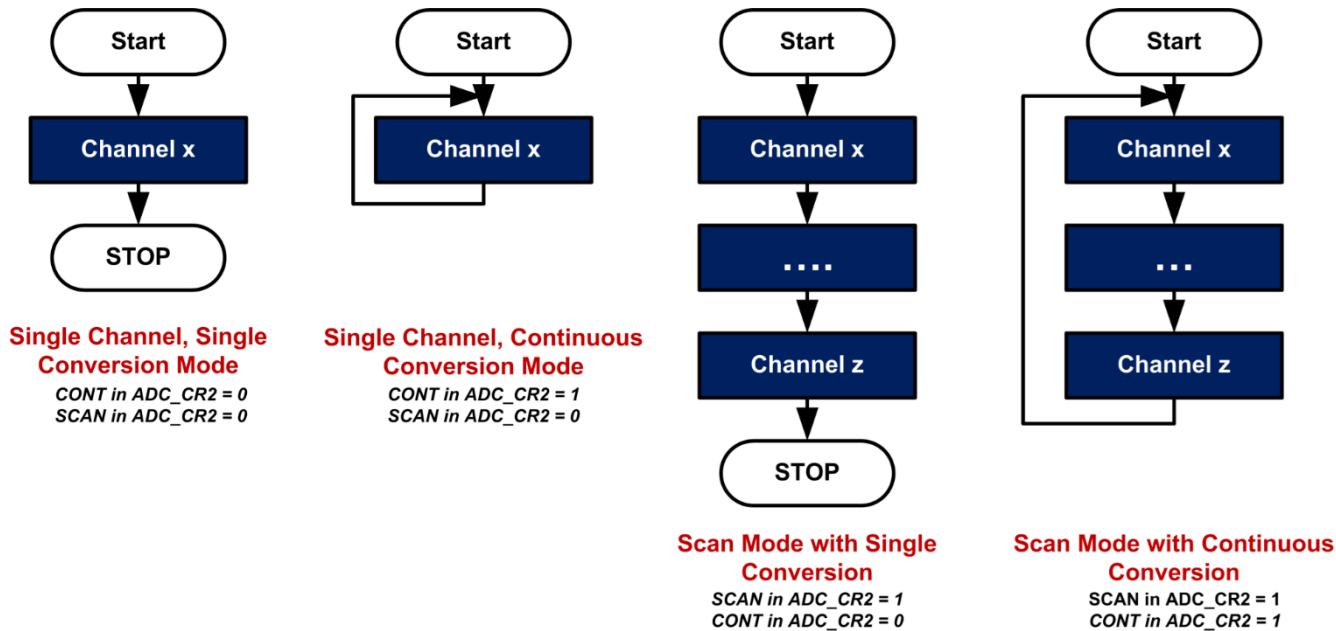
Regular channel:

1. Set SWSTART in ADC_CR2
2. The channel is selected by SQI[4:0] in SQR5
3. Result is stored in ADC_DR
4. EOC is set after conversion
5. Interrupt is generated if EOCIE is set

Injected channel:

1. Set JSWSTART in ADC_CR2
2. The channel is selected by JSQI[4:0] in JSQR
3. Result is stored in ADC_JDR1
4. JEOC is set after conversion
5. Interrupt is generated if JEOCIE is set

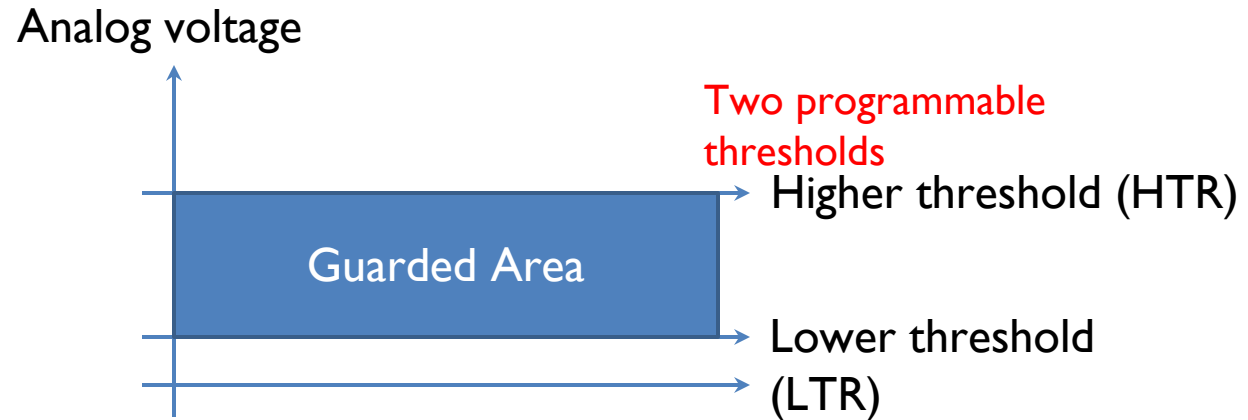
ADC Mode



Channels are selected by **ADC_SQRx** registers for regular channels, and by **ADC_JSQR** register for injected channel

All channels in a regular group share the same result register **ADC_DR**. **Make sure to read data between consecutive sampling.**

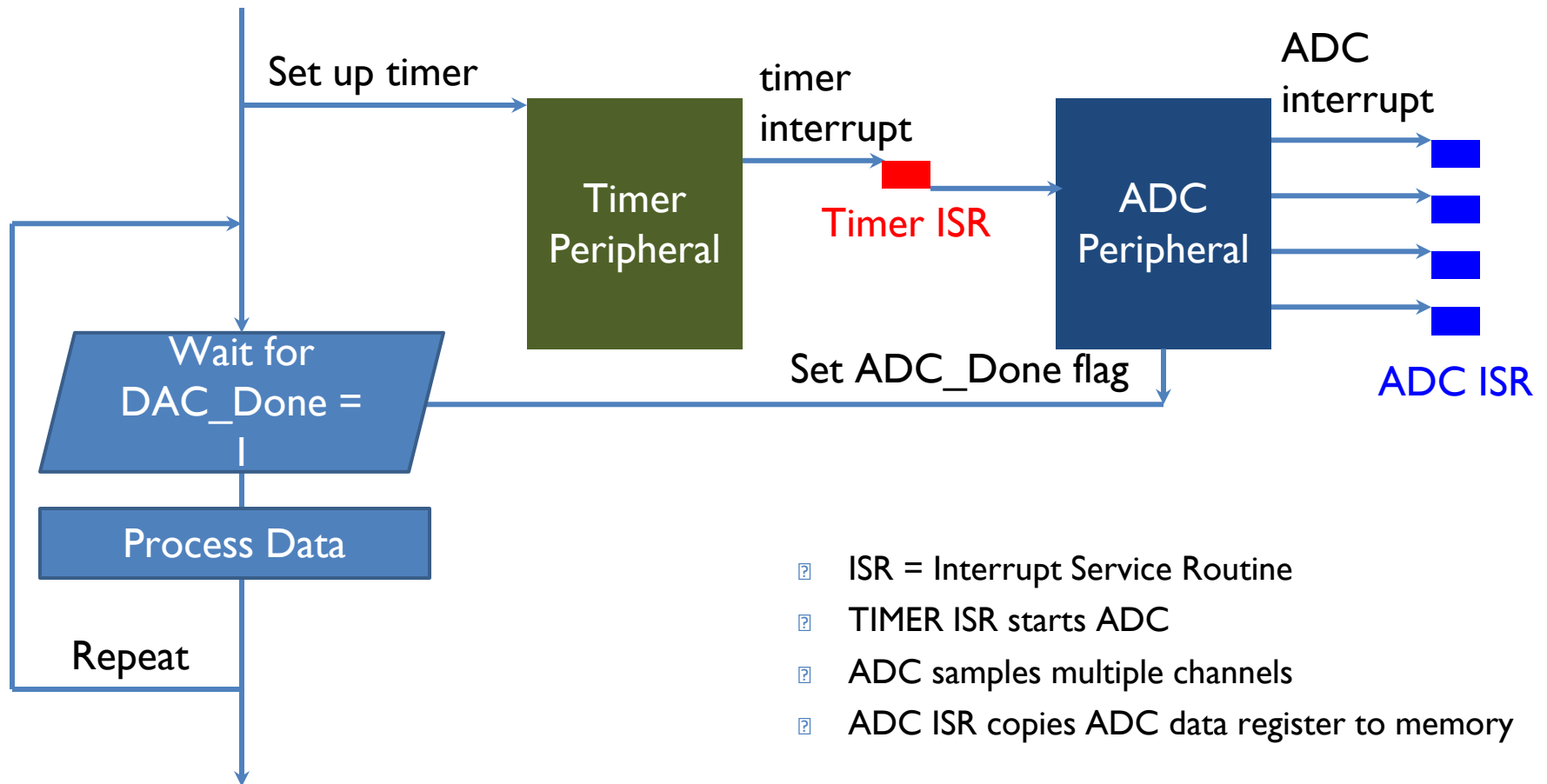
Analog Watchdog



- ❑ If $V < V_{LTR}$ or $V > V_{HTR}$, the analog watchdog (AWD) flag (in ADC Status Register) is set, generating an interrupt to the processor
- ❑ The monitor is automatically performed by hardware, not software
- ❑ Convenient and efficient feature
- ❑ Help processor detect exceptions and recover from specific situations
 - ❑ For example, monitor sensor data and raise alarm on some level.

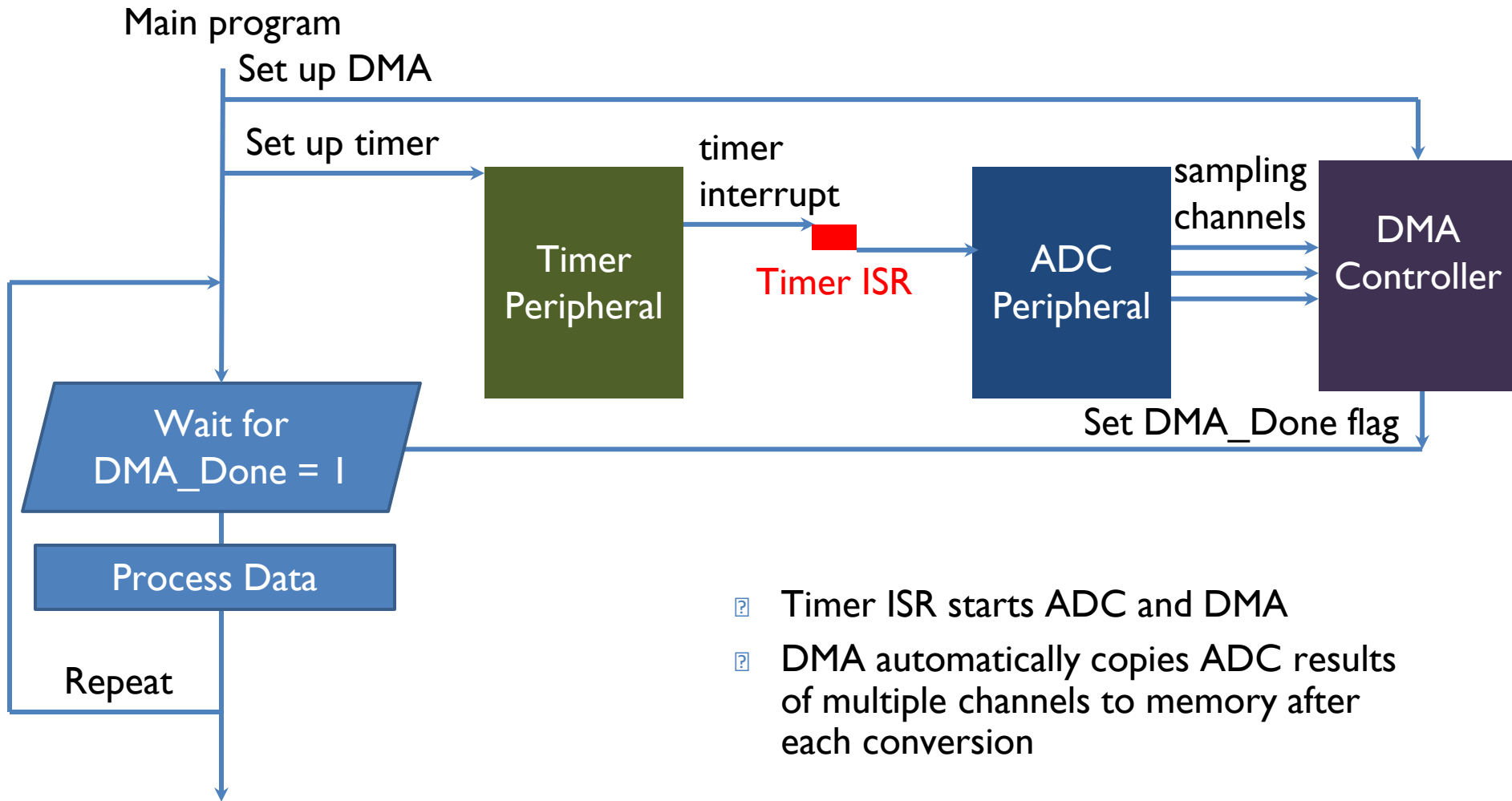
Example: ADC with Timer Interrupts

Main program



- ? ISR = Interrupt Service Routine
- ? TIMER ISR starts ADC
- ? ADC samples multiple channels
- ? ADC ISR copies ADC data register to memory

Example: ADC with Timer and DMA



- ❓ Timer ISR starts ADC and DMA
- ❓ DMA automatically copies ADC results of multiple channels to memory after each conversion